

OBB 与 AABB 碰撞检测：算法流程总结

本文档总结了 OBB (有向包围盒) 与 AABB (轴对齐包围盒) 之间进行碰撞检测的标准算法流程。该算法基于分离轴定理 (Separating Axis Theorem, SAT)。

1. 算法核心思想 (SAT)

分离轴定理指出，如果两个凸体 (OBB 和 AABB 都是凸体) **没有** 发生碰撞，那么**必定**存在一个轴 (称为分离轴)，当两个物体投影到这个轴上时，它们的投影区间互不重叠。

因此，算法的目标不是去寻找重叠，而是去**寻找分离**。

- 算法流程：** 我们测试一系列“潜在的分离轴”。
- 提前退出 (Early-Out)：** 只要在**任何一个**轴上发现分离，我们就可以立即停止计算，并返回 NO_COLLISION。
- 最坏情况：** 如果我们测试了所有可能的轴，**仍然没有**找到分离，那么这两个物体必定处于 COLLISION (碰撞) 状态。

2. 15 个潜在分离轴

对于两个 OBB (AABB 是 OBB 的一种特例) 之间的检测，总共需要测试 **15 个** 潜在的分离轴。

我们定义：

- AABB (A)：** 轴 A_0, A_1, A_2 (即世界坐标系的 X, Y, Z 轴)
- OBB (B)：** 轴 B_0, B_1, B_2 (OBB 的 3 个局部坐标轴)

15 个轴分为三组：

- G1 (3 个轴)：** AABB 的 3 个面法线 (A_0, A_1, A_2)
- G2 (3 个轴)：** OBB 的 3 个面法线 (B_0, B_1, B_2)
- G3 (9 个轴)：** 两个物体 9 对边方向的叉积 (Cross Product)
 - $A_0 \times B_0, A_0 \times B_1, A_0 \times B_2$
 - $A_1 \times B_0, A_1 \times B_1, A_1 \times B_2$
 - $A_2 \times B_0, A_2 \times B_1, A_2 \times B_2$

3. 详细算法流程

算法分为两个阶段：设置阶段 (计算一次) 和测试阶段 (最多 15 次循环)。

阶段 1: 设置 (Setup Phase)

为了高效地进行 15 次测试，我们首先将 OBB B 转换到 AABB A 的局部坐标系中 (即以 AABB 的中心 C_A 为原点，以 A_0, A_1, A_2 为坐标轴)。

1. 输入：

- AABB A : 中心 C_A , 半轴长 $E_A = (e_{A0}, e_{A1}, e_{A2})$
- OBB B : 中心 C_B , 半轴长 $E_B = (e_{B0}, e_{B1}, e_{B2})$, 3 个局部轴 U_0, U_1, U_2

2. 计算相对平移 T ：

- $T = C_B - C_A$ (3 次 FSUB)
- (由于 AABB 轴与世界轴相同, T 在 AABB 局部坐标系中的表示就是 T 本身)

3. 计算相对旋转 R :

- R 是一个 3x3 矩阵, 用于将 B 的轴表示在 A 的坐标系中。
- $R_{ij} = A_i \cdot B_j$
- **关键简化:** 由于 A_i 是世界轴 (如 $A_0 = (1, 0, 0)$), R 矩阵就是 OBB B 的旋转矩阵 U 本身 (或其转置, 取决于约定)。例如, $R_{0j} = A_0 \cdot B_j = (1, 0, 0) \cdot U_j = U_{jx}$ 。
- 这个步骤在操作上是“免费”的 (0 FLOPs), 只是内存访问。

4. 计算 $|R|$:

- 创建一个 3x3 矩阵 `AbsR`, 其元素是 R 矩阵中对应元素的绝对值 (`fabs`)。
- `AbsR_{ij} = |R_{ij}|`。
- 此矩阵将在后续计算中被大量重用。

阶段 2: 轴测试 (15 次, 带提前退出)

这是算法的核心循环。我们按顺序检查 15 个轴。

对于任何给定的轴 L :

1. 计算 AABB A 在 L 上的投影半径 r_A 。
2. 计算 OBB B 在 L 上的投影半径 r_B 。
3. 计算两个中心点在 L 上的投影距离 $d_L = |T \cdot L|$ 。
4. **检查分离:**
 - `if (d_L > r_A + r_B) :`
 - `return NO_COLLISION (提前退出)`

轴组 1: AABB 的 3 个轴 (G1)

- **测试 $L = A_0$ (即 X 轴):**
 - $r_A = e_{A0}$
 - $r_B = e_{B0} \cdot AbsR_{00} + e_{B1} \cdot AbsR_{01} + e_{B2} \cdot AbsR_{02}$
 - $d_L = |T_x|$
 - `if (|T_x| > r_A + r_B) return NO_COLLISION;`
- **测试 $L = A_1$ (即 Y 轴):**
 - (逻辑同上, 使用 T_y, e_{A1} 和 R 的第 1 行)
 - `if (|T_y| > ... ) return NO_COLLISION;`
- **测试 $L = A_2$ (即 Z 轴):**
 - (逻辑同上, 使用 T_z, e_{A2} 和 R 的第 2 行)
 - `if (|T_z| > ... ) return NO_COLLISION;`

轴组 2: OBB 的 3 个轴 (G2)

- 测试 $L = B_0$ (即 R 的第 0 列):
 - $r_A = e_{A0} \cdot AbsR_{00} + e_{A1} \cdot AbsR_{10} + e_{A2} \cdot AbsR_{20}$
 - $r_B = e_{B0}$
 - $d_L = |T_x \cdot R_{00} + T_y \cdot R_{10} + T_z \cdot R_{20}|$ (即 $T \cdot L$)
 - if (d_L > r_A + r_B) return NO_COLLISION;
- 测试 $L = B_1$ 和 $L = B_2$:
 - (逻辑同上, 使用 R 的第 1 列和第 2 列)
 - if (...) return NO_COLLISION;
 - if (...) return NO_COLLISION;

轴组 3: 9 个叉积轴 (G3)

- 测试 $L = A_0 \times B_0$:
 - (L 和 r_A, r_B, d_L 都可以根据 T 和 R 的分量预先计算出来, 以避免运行时的叉积和点积)
 - if (d_L > r_A + r_B) return NO_COLLISION;
- 测试 $L = A_0 \times B_1$:
 - if (...) return NO_COLLISION;
- ... (继续测试剩下的 7 个叉积轴) ...
- 测试 $L = A_2 \times B_2$:
 - if (...) return NO_COLLISION;

阶段 3: 最终结论

- return COLLISION
- 分析: 如果算法执行到这一步, 意味着它已经测试了所有 15 个潜在的分离轴, 但**没有一个**能够成功分离这两个物体。根据 SAT, 它们必定正在发生碰撞。